Intelligent Long-Term Memory for Conversational AI

A local prototype measured against vector search

Franco Aseglio

November 2026

https://github.com/FrancoAseglio/llm-mem-sys

Contents

1 Introduction **1**

2 Task Breakdown **1**

2.1 Step 1: Storage and Write Path . . . 1

2.1.1 Design and its motivations . . . 1

2.1.2 Issues . . . 1

2.1.3 Requirements . . . 1

2.1.4 Limitations . . . 2

2.2 Step 2: Read Path . . . 2

2.2.1 Design and its motivations . . . 2

2.2.2 Issues . . . 2

2.2.3 Requirements . . . 2

2.2.4 Limitations . . . 3

2.3 Step 3: Summaries . . . 3

2.3.1 Design and its motivations . . . 3

2.3.2 Issues . . . 3

2.3.3 Requirements . . . 3

2.3.4 Limitations . . . 3

2.4 Step 4: Evaluation . . . 3

2.4.1 Design and its motivations . . . 3

2.4.2 Issues . . . 3

2.4.3 Requirements . . . 4

2.4.4 Limitations . . . 4

3 Conclusions **4**

4 Extra Information **5**

4.1 Division of Work . . . 5

4.2 Future Work Propositions . . . 5

5 Bibliography **5**

1 Introduction

This project's goal was to design and build a long-term memory for an LLM assistant that stores what users say over long periods, prioritises relevant information, detects contradictory facts, updates outdated ones, compresses old history into summaries and keeps answers consistent; the expected outcome was a prototype that answers better than traditional vector memory, where past messages are stored and the most similar ones are added to the prompt.
Everything runs on one machine: Python, Postgres with `pgvector`, and Ollama serving `llama3.1:8b` [5] for extraction and answers and `nomic-embed-text` [6] for embeddings.
The design draws on the memory stream and relevance, recency and importance scoring of generative agents [1] and the tiered context of MemGPT [2], and is close in spirit to production memory layers that extract and update facts [3] or keep a temporal knowledge graph [4].

The repository is organised into:

- **src/memory_system**: the memory itself, split into `core`, `write`, `read` and `summaries`;
- **evaluation**: the test sets (scripted stories with questions), the runners and the scoring; every run is kept as CSV tables in **results**;
- **notebooks**, **dashboard**: charts and a local web page to browse questions, answer and to query the memory;
- **docs**, **report**: the design, the evaluation and this document.

2 Task Breakdown

2.1 Step 1: Storage and Write Path

2.1.1 Design and its motivations

Memory is kept in three layers of one Postgres database: the **messages**, untouched; structured **facts** extracted from them (`lives_in = Milan`); and **episode notes** (Step 3). Each user message goes through a write path:

- **Extraction**: the local model returns facts as JSON matching a schema, at temperature 0, each of one of 13 relation types marked *single* (city, employer) or *multi* (hobbies, allergies), and each an *assert* or a *retract*.
- **Pure checks**: the value must appear in the message (*grounding*), be about the user rather than a sister or a friend, and a new city, employer or partner must come with words that signal a change (*change rule*).
- **Versioning, never overwriting**: a new single value makes the old one `superseded`; a retraction makes it `archived`; a duplicate raises confidence; a like and a dislike of the same thing contradict each other; a job title becomes `conflicted` when the employer changes. Every decision is logged in an audit table.

Keeping history instead of overwriting it was motivated by the questions about the past ("Where did I live before Oslo?") and by the need to explain every behaviour from the log.

2.1.2 Issues

- **Extraction errors**: the 8B model filed other people's facts under the user (a sister's city), turned incidental mentions into facts and slipped on corrections. On the first set of unseen data, half of the misses of facts-only memory came from facts stored wrongly.
- **Incidental replacements**: "looking at houses in Rotterdam, prices in Utrecht are crazy" moved a user to another city, since a single-valued fact is replaced by any new value.
- **Brittle word lists**: the change rule, written on the development sets, kept only 8 of 12 key facts right when changes were worded differently in the final blind set.

2.1.3 Requirements

The step meets three of the project goals:

- **Long-term storage**: messages, versioned facts with validity dates, notes and an audit log of 11 event types, in one schema (`db/schema.sql`).
- **Contradiction detection**: likes against dislikes, retractions, and a job title flagged as `conflicted` after an employer change.
- **Updating outdated information**: replaced values are superseded with an end date and never reach the prompt as current; on the first six sets an outdated value reached the prompt 15 times with vector memory and 0 times with facts.

2.1.4 Limitations

- **A fixed schema**: information outside the 13 relation types is dropped or misfiled.
- **Rules in English**: grounding, the change rule and the chatter filter rely on English word lists, which do not carry over to new wording.
- **The `relations` table** meant for a small knowledge graph is unused; typed facts play its role.

2.2 Step 2: Read Path

2.2.1 Design and its motivations

Similarity search always returns something, so the first design replaced ranking with a **relevance gate**: a fact reaches the prompt only if its similarity to the question is at least 0.42 (*floor*), within 0.08 of the best fact (*window*), and its relation type is among the two that best match the question (*routing*); allergies skip the floor and window when routing picks them. What passes is ranked by a weighted score (similarity, importance, usage, confidence, recency) and cut to four lines, so the prompt can stay empty. Questions about the past also read superseded facts.

After the facts-only design lost on long histories (Figure 1), the final **hybrid** mode keeps the raw messages and uses facts as annotations: the current facts of the two best relation types, one timeline for a past question (`Lives in, oldest first: Stavanger -> Oslo -> Kristiansand (now)`), then the four most similar messages, each marked when a fact it gave has changed: `"I've moved to Oslo" [outdated: Lives in: Oslo -> now Kristiansand]`.

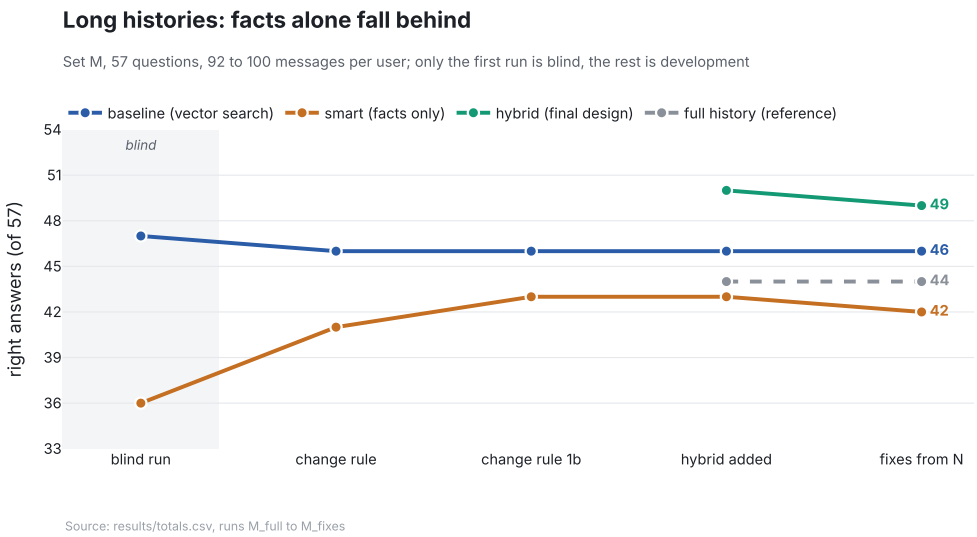


Figure 1: Set M (57 questions, users with 92 to 100 messages): facts alone stay below vector search through every fix; hybrid, added afterwards, overtakes it. Only the first run is blind.

2.2.2 Issues

- **No single threshold works**: a topical neighbour (`lives_in = Rome`, 0.50) can outscore a needed fact for another question (an allergy at 0.44), hence the window and routing on top of the floor.
- **Routing vetoes strong matches**: `goal = learn Japanese` at 0.70 similarity was dropped because its type ranked fourth for "Which language am I trying to learn?".
- **Facts lose to messages on long histories**: on set M, vector search answered 47 questions and facts with notes 36; the raw sentences carry details that extraction drops.

2.2.3 Requirements

- **Prioritising relevant information**: irrelevant text reached 14 of 46 prompts instead of 42, with one or two memory lines instead of four.
- **Consistency**: outdated values are either absent (facts) or labelled (hybrid), so the model is not left to choose between "I live in Turin" and "I moved to Milan".

2.2.4 Limitations

- **Tuned on seen data**: the floor, the window and two routing hints were chosen on the first three sets; every set is now spent, so retuning needs a new blind set.
- **A marker is only as right as its fact**: a wrong supersession marks a true message as outdated, and only the message that first gave a fact is marked.
- **Longer prompts**: hybrid reads about five lines per question against the baseline's four.

2.3 Step 3: Summaries

2.3.1 Design and its motivations

Facts cannot hold episodes, plans or purchases ("I baked a loaf and named it Bruno").
Every few messages, the ones not yet seen are sent to the model, which writes one-sentence **episode notes**, each citing its source messages; a faithfulness check drops a note whose names, numbers or most of its words are not in those messages. Notes older than 30 days are merged by topic into **digests**, checked again (faithful, no name or number lost, shorter), which replace them in retrieval; nothing is deleted.

2.3.2 Issues

- **Hallucinated or chatty notes**: the model wrote notes about questions and thanks, so a code filter removes them before the call, and invented details, hence the check.
- **Runaway generations**: a structured call once generated without end;
  every model call now has a length cap and a timeout.
- **Little compression**: notes restate their sources in the third person, about as long as the messages they came from.

2.3.3 Requirements

Compressing history is *partly* met:
notes recovered most questions about episodes (needed information present for 13 of 17 instead of 8), but digests were 0.79 of the notes they replace against a pre-registered target of 0.70.

2.3.4 Limitations

- **Faithful words, wrong meaning**: the check is word-level, so a misattribution made of source words passes.
- **Details versus length**: keeping every name and number caps the saving; no message is archived, so the memory still grows with every message.

2.4 Step 4: Evaluation

2.4.1 Design and its motivations

Fifteen sets of scripted life stories (86 scenarios, 367 questions) are replayed message by message with backdated timestamps, then every question is asked with no memory, vector search over the four most similar messages (the **baseline**), facts only (**smart**), **hybrid**, and the **full** history as a reference.
Prompts and answers are scored by keywords: needed facts present, noise, outdated values, right answers.
To keep the measurement honest:

- **Predictions first**: written and committed before every run of a new set.
- **Blind sets**: written before the change they test, run once, then *spent*.
- **Read-only measurement** and **deterministic calls** (temperature 0, cached).

2.4.2 Issues

- **Running out of fresh data**: every tuning step spends a set, which is why the final claim rests on two sets written after the design was frozen.
- **Crude scoring**: a right answer worded differently counts as wrong; two counts on set M were reviewed by hand.

2.4.3 Requirements

The evaluation follows the plan of the proposal:

- **Baseline**: the same model with plain vector memory (the four most similar messages).
- **Test data**: 86 scripted life stories instead of 20 to 50, with moves, job changes, changed preferences and corrections.
- **Measures**: answer accuracy, outdated values in prompts and answers, retrieval precision (needed facts and noise), prompt size, and consistency across sessions (set K). Latency and human ratings were not measured; keyword checks replaced the ratings.
- **Success criterion**: *clearly better on accuracy, staleness and consistency, with fewer prompt tokens.* Not met. On the blind sets N and O (114 questions, Figure 2) hybrid answered 97 against the baseline's 89, meeting the pre-registered rule of +3 on each set, but not significantly: hybrid alone was right on 15 questions and the baseline alone on 7 ($p = 0.13$). Outdated answers were level (7 against 6) and hybrid's prompts were longer (5.2 lines against 4.0). Facts help on values before a change (86% against 57%) and on indirect questions such as allergies (60% against 10%), Figure 3.

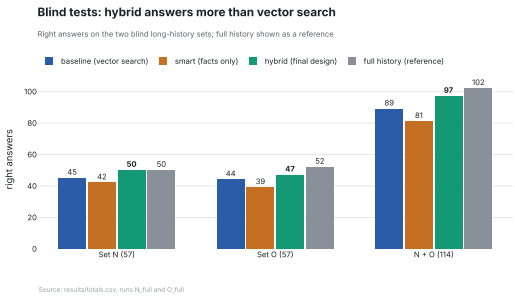


Figure 2: Right answers on blind sets N and O.

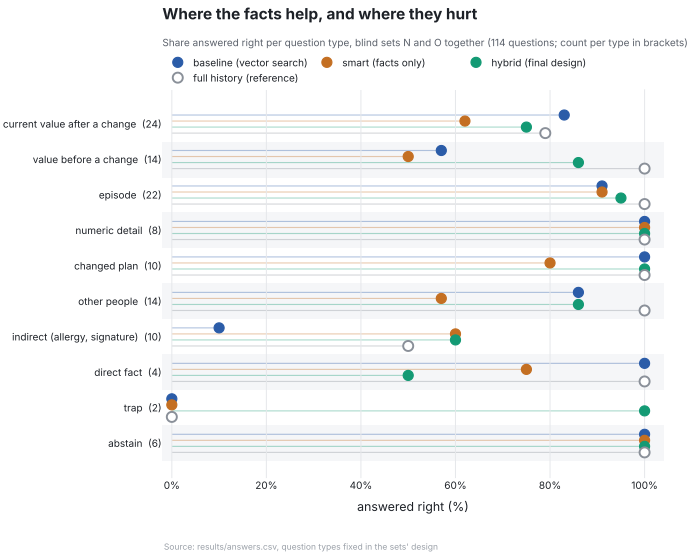


Figure 3: Share answered right per question type, N and O.

2.4.4 Limitations

- **The full history scored best** (102): about 1,100 tokens per user fit the model whole, so these sets show memory beating vector search, not beating the full context.
- **Small and self-written**: six long histories carry the headline, the scenarios share a designer with the system, and one small model was used throughout.

3 Conclusions

- **Facts as annotations, not replacements**: structured facts are good at what changes (current values, timelines, allergies) and bad at details; marking outdated messages kept the strengths of both.
- **Extraction is the bottleneck**: almost every loss of the fact layers traces back to a fact stored wrongly by the 8B model, not to retrieval.
- **Honest measurement changes the design**: blind sets and written predictions turned a promising early result (57 against 52 on the first sets) into a rejected hypothesis on long histories, and that rejection led to the final design.
- **Memory must prove itself at length**: at the history lengths tested, putting everything in the prompt still wins; memory pays off only beyond the context window.

4 Extra Information

The project is a Poetry package and runs locally.
Everything except Ollama runs in Docker: `make up` starts Postgres, preloaded with the evaluation users, and the dashboard at `localhost:8000`; Ollama stays on the host, since Docker on macOS cannot use the GPU. `make help` lists every command, `make check` runs the 352 tests, and the detailed development documents and predictions are kept at the git tag `docs-archive`.

4.1 Division of Work

The project was carried out individually: design, test scenarios, predictions, experiments and their analysis. An AI assistant (Claude, Anthropic) was used as a programming partner, mainly for the evaluation tooling. The predictions for every test set were committed before its run, as the repository history shows.

4.2 Future Work Propositions

- **Longer histories**: a blind set beyond the context window, where full history is no longer possible and memory has to prove its worth.
- **A stronger extractor**: a larger model for extraction only, to test whether the hand-written rules are still needed.
- **Retrieval follow-ups**: a strong-match override for routing, measured as a pre-registered ablation; LLM-judged scoring next to keywords.

5 Bibliography

References

[1] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, M. S. Bernstein. *Generative Agents: Interactive Simulacra of Human Behavior.* UIST 2023. arXiv:2304.03442.

[2] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, J. E. Gonzalez. *MemGPT: Towards LLMs as Operating Systems.* arXiv:2310.08560, 2023.

[3] P. Chhikara, D. Khant, S. Aryan, T. Singh, D. Yadav. *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory.* arXiv:2504.19413, 2025.

[4] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, D. Chalef. *Zep: A Temporal Knowledge Graph Architecture for Agent Memory.* arXiv:2501.13956, 2025.

[5] A. Grattafiori et al. *The Llama 3 Herd of Models.* arXiv:2407.21783, 2024.

[6] Z. Nussbaum, J. X. Morris, B. Duderstadt, A. Mulyar. *Nomic Embed: Training a Reproducible Long Context Text Embedder.* Transactions on Machine Learning Research, 2025. arXiv:2402.01613.